

Q1 RFLAGS

1 Point

Suppose `%rdi=0x1` and `%rsi=0x2`, which status flag is set after instruction `cmp %rsi, %rdi`?

ZF

SF

CF

OF

$$\%rdi - \%rsi$$

$$= 0x1 - 0x2$$

$$= 0xffffffff \dots ff$$

overflow for unsigned

not overflow for signed ($1 - 2 = -1$)

Q2 cmp and set instructions

4 Points

For the C function below:

Handwritten annotations: %rdi points to x, %rsi points to y

```
bool foo(long x, long y)
{
    return x > y (1);
}
```

Its compiled x86 assembly is the following:

```
foo:
    cmp %rsi, %rdi → x - y
    setg %al → result > 0?
    ret
```

Q2.1 function arguments

1 Point

Which register stores foo's first argument ? (For auto-grading, prefix register name with %)

%rdi

Q2.2 function arguments

1 Point

Which register stores foo's second argument ? (For auto-grading, prefix register name with %)

%rsi

Q2.3

1 Point

What could be the C expression in blank (1)?

✓ `x > y`

`x >= y`

`y > x`

`y >= x`

`x == y`

`x != y`

Q2.4

1 Point

Suppose function arguments have changed to be of the type `unsigned long x` and `unsigned long y`. Then which of the following statements are true about the function's compiled assembly?

It remains the same as before.

The `setg %al` instruction is changed to `setl %al`

The `setg %al` instruction is changed to `seta %al`

The `setg %al` instruction is changed to `setb %al`

Q3 Control flow and memory reference mode

6 Points

Below is an incomplete C function called `foo`. `foo` takes one argument `s` whose type we'll try to infer later:

```
1. long foo(char* s) {  
2:   long cnt = 0; s[i] != 0 i++  
3:   for (long i = 0; (1); (2)) {  
4:     cnt++ (3);  
5:   }  
4:   return cnt;  
5: }
```

Below is the x86 assembly for the disassembled function `foo`:

```
movq $0x0, %rax → return value : cnt  
movq $0x0, %rdx → i  
.L2:  
  cmpb $0x0, (%rdi, %rdx, 1) ⇒ element in s is 1 byte  
  je .L1  
  addq $0x1, %rdx  
  addq $0x1, %rax  
  jmp .L2  
.L1:  
  retq
```

Q3.1

1 Point

Suppose `%rdi` has value `0x0000deadbeef0000` and `%rdx` has value `2`, which of the following statements is true of instruction `cmpb $0x0, (%rdi, %rdx, 1)` $\%rdi + \%rdx * 1$

It compares 4-byte constant `0x0` with 4-byte value `0x0000deadbeef0002`

It compares 1-byte constant `0x0` with 1-byte value `0x02`

It compares 4-byte constant `0x0` with the 4-byte value stored at address `0x0000deadbeef0002`

✓ It compares 1-byte constant `0x0` with the 1-byte value stored at address `0x0000deadbeef0002`

It compares 4-byte constant `0x0` with the 4-byte value stored at address `0x0000deadbeef0000`

It compares 1-byte constant `0x0` with the 1-byte value stored at address `0x0000deadbeef0000`

Q3.2

1 Point

Where is the loop variable `i` stored at?

`%rax`

✓ `%rdx`

`%rdi`

stored in memory under the address given by `%rdi+%rdx`

Q3.3

1 Point

What is the most likely type of function argument `s`?

`long`

`int`

`char`

`long *`

`int *`

✓ `char *`

Q3.4

1 Point

The instruction `cmpb $0x0, (%rdi, %rdx, 1)` sets RFLAGS according to the subtraction result: `??? - 0`

What is a valid C expression for `???`?

`s[1]`

✓ `s[i]`

`s+i*4`

`s+i*8`

Q3.5**1 Point**

Please fill in the blank (1) in the C code.

For autograding purpose, adhere to the rules below:

- use your answer in Q3.4 in filling the blank.
- keep a constant, if there is one, on the ride-hand-side of any binary operator
- use `0` instead of `'\0'` to refer to ascii null character
- no space character

```
s[i]!=0
```

Q3.6**1 Point**

Please fill in the blank (2) in the C code. To facilitate autograding, please do not include the trailing `;` in your answer.

```
i++
```